

UNINASSAU OLINDA

DISCIPLINA: ARQUITETURA DE SOFTWARE

DOCUMENTAÇÃO TÉCNICA DO PROJETO AgendaSUS E SUA APLICAÇÃO EM ARQUITETURA DE SOFTWARE

**AgendaSUS - SISTEMA PÚBLICO DE AGENDAMENTO
HOSPITALAR**

OLINDA, 2026

AgendaSUS - SISTEMA PÚBLICO DE AGENDAMENTO HOSPITALAR

DANIEL DANTAS DE FARIAS - 01254056

ERICLYS SEVERINO DA SILVA - 01656023

MÁRCIO JOSÉ DA SILVA MORAES JUNIOR - 01813686

OTHON HENRIQUE QUEIROZ DE OLIVEIRA - 01809897

RODRIGO NUNES PEIXOTO RAMOS - 01777171

OLINDA, 2026

Resumo

Este documento apresenta o protótipo arquitetural do projeto **AgendaSUS**, um sistema web voltado para a otimização e gerenciamento do fluxo de atendimentos na rede pública de saúde da cidade do Recife - PE. O sistema visa mitigar os gargalos históricos de marcação de consultas e exames, oferecendo funcionalidades para filtrar atendimentos por especialidade, realizar agendamentos, monitorar consultas ativas e mapear as clínicas e unidades de saúde municipais. Alinhado aos conceitos de governança digital e eficiência urbana, a proposta tecnológica busca unificar e descentralizar o acesso às informações de saúde, estimulando a transparência e a agilidade no atendimento ao cidadão recifense.

1. Introdução

O Sistema Único de Saúde (SUS) na cidade do Recife enfrenta desafios crônicos relacionados à gestão de filas, descentralização de informações e acessibilidade aos serviços de agendamento. Pacientes frequentemente enfrentam longas esperas presenciais ou dificuldades de comunicação para confirmar, cancelar ou verificar a disponibilidade de consultas e exames nas clínicas e unidades básicas da capital pernambucana. Esses obstáculos geram absenteísmo (não comparecimento às consultas), sobrecarga nos postos de atendimento físico e ineficiência na alocação de recursos médicos municipais.

Com o avanço das tecnologias de informação e comunicação aplicadas à saúde pública (e-Saúde), torna-se imperativo o desenvolvimento de soluções que aproximem o cidadão das redes de atendimento. A ausência de uma interface centralizada, intuitiva e responsiva que conecte o usuário diretamente à escala de especialidades médicas compromete o planejamento estratégico da gestão de saúde do município. Diante desse cenário de colapso informacional e necessidade de modernização administrativa, faz-se necessária uma intervenção digital estruturada.

2. Proposta

O **AgendaSUS** apresenta-se como uma solução tecnológica modular e centralizada, projetada para atuar como um mediador digital entre os cidadãos recifenses e a infraestrutura de saúde pública municipal. O objetivo principal da plataforma é simplificar a

jornada do paciente, fornecendo um ambiente digital transparente e acessível para a gestão autônoma de sua saúde preventiva e emergencial.

A plataforma estruturar-se-á em torno de quatro pilares funcionais:

- **Filtro Avançado de Consultas:** Permite a busca detalhada por especialidades médicas, proximidade geográfica e disponibilidade de horários na rede municipal.
- **Agendamento Dinâmico:** Interface simplificada para a marcação de consultas de forma remota, reduzindo a necessidade de deslocamento prévio até as unidades de saúde.
- **Módulo de Verificação de Agendamentos:** Um painel centralizado onde o usuário pode acompanhar o status de suas solicitações, receber lembretes e realizar cancelamentos prévios, otimizando a taxa de ocupação das vagas médicas.
- **Mapeamento de Clínicas e Unidades:** Listagem dos pontos de atendimento ativos em Recife.

A proposta foca estritamente na acessibilidade e inclusão digital, garantindo que cidadãos de diferentes faixas etárias ou níveis de familiaridade tecnológica consigam navegar com facilidade pela plataforma, promovendo a democratização do acesso à saúde.

3. Metodologia

A metodologia de desenvolvimento do protótipo adota uma abordagem de engenharia de software modular e containerizada, garantindo a separação de responsabilidades entre a interface do usuário, a lógica de negócios e o armazenamento de dados. O projeto está estruturado a partir dos seguintes componentes tecnológicos:

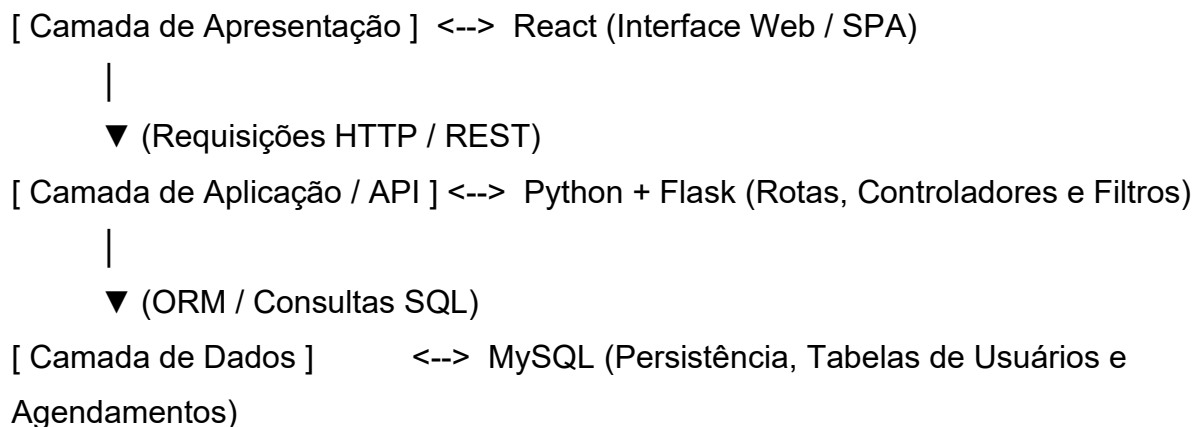
1. **Frontend (Camada de Apresentação):** Construído utilizando a biblioteca React. A interface será projetada focando nos princípios de UX/UI (Experiência e Interface do Usuário), com design responsivo para funcionar perfeitamente em dispositivos móveis e desktops, garantindo clareza visual e facilidade de navegação para o cidadão.

2. **Backend (Camada de Lógica de Negócios):** Desenvolvido em linguagem Python utilizando o micro-framework Flask. Esta camada será responsável por expor uma API RESTful estruturada, gerenciando a autenticação de usuários, validação das regras de negócio de agendamento (evitando marcações duplicadas) e a comunicação com a camada de persistência.
3. **Banco de Dados (Camada de Persistência):** Utilização do sistema de gerenciamento de banco de dados MySQL. A estrutura relacional garantirá a integridade e a consistência dos dados confidenciais dos pacientes, histórico de consultas, escalas médicas e catálogo de clínicas da rede pública de Recife.
4. **Ambiente de Desenvolvimento e Deploy (Infraestrutura):** Toda a aplicação será encapsulada e executada através da tecnologia Docker. A abordagem adota a estratégia de container monolítico (*All-in-One*), unificando o build de produção do frontend (React), o ambiente de execução do backend (Flask) e o servidor do banco de dados (MySQL) dentro de uma única imagem isolada.

4. Arquitetura

O padrão de arquitetura de software adotado para o **AgendaSUS** baseia-se no modelo MVC (Model-View-Controller). Essa abordagem organiza o sistema dividindo-o em três componentes fundamentais e interligados, o que separa a representação interna das informações da forma como são apresentadas e aceitas pelo usuário.

A estrutura é segmentada da seguinte forma:



A escolha deste modelo arquitetural justifica-se pelos seguintes fatores:

- **Separação de Conceitos (Divisão):** Desacopla completamente a interface (React) das regras de agendamento e segurança (Flask), permitindo que modificações visuais não interfiram na integridade dos dados de saúde pública.
- **Desenvolvimento Incremental:** Visando futuras expansões do sistema (como integração com prontuários eletrônicos nacionais ou APIs do Ministério da Saúde), novas funcionalidades podem ser acopladas em camadas específicas sem comprometer o núcleo estável da aplicação.
- **Facilidade de Deploy com Docker:** A centralização da infraestrutura em um único container unifica a interface Web, a API Flask e o banco de dados MySQL no mesmo ciclo de vida. Essa abordagem simplifica drasticamente o deploy, eliminando a complexidade de orquestrar múltiplos microsserviços e isolando completamente toda a aplicação de falhas ou inconsistências no sistema operacional do servidor.

5. Componentes

O Blueprint é uma ferramenta essencial para organizar aplicações web. Ele permite dividir um sistema grande em módulos independentes e bem isolados, em vez de deixar todas as rotas concentradas em um único arquivo, podendo assim organizar o projeto por áreas de negócio como agendamentos, usuários ou clínicas.

Na prática, ele promove o desacoplamento, facilita a divisão clara de responsabilidades no padrão MVC e permite que vários desenvolvedores trabalhem ao mesmo tempo sem pisar no pé uns dos outros. Além disso, funciona como um componente plugável: você registra os blueprints na aplicação principal e pode aplicar configurações em lote, como prefixos de URL, mantendo a arquitetura limpa e profissional.

- `App.py`
- `from controllers.appointment_controller import appointment_bp`

- `@app.route('/')`
- `def index():`
- `return render_template('index.html')`
- `app.register_blueprint(appointment_bp, url_prefix='/api')`
- `Appointment_controller.py`
- `from flask import jsonify, request, Blueprint`
- `appointment_bp = Blueprint('appointment', _name_`

6. Ferramentas de Desenvolvimento

Para garantir a confiabilidade, estabilidade e evolução contínua da API do **AgendaSUS**, foi integrada ao projeto uma suíte de testes automatizados utilizando o framework Pytest em conjunto com a biblioteca nativa Unittest.mock.

A escolha dessa abordagem arquitetural justifica-se pelos seguintes fatores:

- **Isolamento de Componentes (Mocking):** Em vez de depender do estado mutável de um banco de dados MySQL físico (o que causaria falhas por registros duplicados ou dados ausentes), os testes utilizam *Mocks* (simulações) para interceptar as chamadas aos modelos e forçar comportamentos controlados.
- **Velocidade e Automação:** Permite validar todas as rotas e regras de comunicação da aplicação em milissegundos a cada modificação do código, prevenindo regressões de software antes do deploy em contêineres Docker.
- **Garantia de Contratos de API:** Certifica que os controladores retornam exatamente os códigos de status HTTP (como 200 OK ou 201 Created) e as estruturas JSON que a camada de apresentação em React espera receber.

Abaixo estão os 6 testes de integração implementados na pasta tests/e2e/ da aplicação:

1. **test_index_route (test_app.py)**

- **Objetivo:** Validar a inicialização da infraestrutura do Flask e o roteamento da página inicial.
- **Explicação:** Garante que a rota raiz (/) está devidamente configurada para carregar a camada de apresentação do sistema.

2. **test_get_users_api (test_users.py)**

- **Objetivo:** Validar o endpoint de listagem de usuários e pacientes do sistema.
- **Explicação:** Certifica que a rota /api/users responde com sucesso (Status 200) e está pronta para enviar os dados de cidadãos cadastrados no sistema de saúde.

3. **test_create_appointment_success (test_appointments.py)**

- **Objetivo:** Validar a regra de negócio de criação de um novo agendamento hospitalar via método HTTP POST.
- **Explicação:** Envia uma requisição contendo as chaves obrigatórias (patient_id e free_schedule_id). Utiliza um *Mock* para isolar o banco de dados e confirma se o controlador processa a ação, devolvendo o status de criação correto (210 / 201 Created) e a mensagem de sucesso.

4. **test_get_specialties_data_success (test_specialties.py)**

- **Objetivo:** Validar a rota de recuperação do catálogo de especialidades médicas.
- **Explicação:** Simula o retorno de uma lista estruturada de especialidades (como Cardiologia e Pediatria) na rota /api/specialties, garantindo que a API entregue um payload JSON legível para alimentar os filtros do frontend.

5. **test_get_free_schedules_success (test_schedules.py)**

- **Objetivo:** Validar a rota geral de busca de horários abertos para agendamento.
- **Explicação:** Testa o endpoint /api/free_schedules para garantir que o sistema consegue expor corretamente em formato de lista os horários de consultas médicas que ainda estão disponíveis na rede municipal.

6. **test_get_filtered_schedules_search & test_get_patient_schedules_search (test_schedules.py)**

- **Objetivo:** Validar as regras de filtragem dinâmica e histórico de consultas por paciente.
- **Explicação:** Garante que os endpoints `/api/filtered_schedules/search` e `/api/schedule_by_patient/search` conseguem interceptar parâmetros enviados na URL (como filtros de data, especialidade ou ID do paciente) e responder adequadamente de forma dinâmica.

7. Implantação

Esta seção descreve a estratégia de containerização do AgendaSUS, justificando as decisões arquiteturais e o processo de deploy.

A escolha pela tecnologia do **Docker** para o deploy do AgendaSUS baseou-se nos seguintes pilares:

- **Isolamento de Ambiente Global:** Toda a infraestrutura da aplicação — incluindo as dependências do Frontend (React/Node.js), do Backend (Flask/Python) e o Banco de Dados — está isolada de ponta a ponta dentro de um único container. Isso garante que nenhuma alteração ou atualização no sistema operacional do servidor afete o funcionamento do sistema.
- **Consistência Absoluta (DevOps):** Garante que o ambiente de desenvolvimento e produção sejam idênticos. O build do frontend e as configurações do banco são executados de forma idêntica em qualquer máquina que rode o container, eliminando divergências de versões de pacotes.
- **Deploy Simplificado (Single Container):** Ao centralizar o ecossistema no mesmo ciclo de vida através de um único Dockerfile, o processo de implantação torna-se extremamente ágil, exigindo o gerenciamento de apenas uma única imagem e container.
- **Automação de Inicialização:** O container é configurado para, ao iniciar, realizar o build dos arquivos estáticos do React, levantar o serviço da API Flask e inicializar o banco de dados com suas respectivas tabelas, chaves estrangeiras e *triggers* de validação de horários.